

WISN Core: Beginner's Project Walkthrough

A two-day teaching guide, developer walkthrough, and evaluator Q&A

Audience: A first-time presenter with little or no Laravel experience

Purpose: Teach what the project does, how its code and data fit together, what technologies it uses, and how to answer likely academic questions.

Repository basis: [niranjanrimal25/wisn-core](#), reviewed against commit [51e5d75](#) on 5 October 2026.

Use this guide honestly. It describes the code that is in this repository. It separates the annual WISN planning calculation from the newer, manual shift-operations prototype. It does not claim that the application is connected to a hospital EHR, contains real patient records, or automatically assigns nurses.

PROJECT STORY

What it solves and where it fits

TECH STACK

What each tool does in plain language

DEFENSE PREP

Two-day lessons and evaluator answers

Guide contents

A teaching path from the project story to the code and likely defense questions.

1. How to use this guide	3
2. A two-day crash-course plan	3
3. The project in one minute	4
4. WISN explained for a beginner	4
5. The technology stack	6
6. Architecture using a simple analogy	7
7. Database: what is stored and how it relates	7
8. WISN calculation: step by step	9
9. Walkthrough of the main user journeys	10
10. The operational staffing overlay: current behavior	11
11. Where the information came from	12
12. Start-to-finish teaching walkthrough: exactly what to open and say	13
13. How to run a local demonstration	17
14. Security, testing, and quality questions	18
15. Evaluator questions and ready-to-say answers	18
16. Short presentation script	22
17. Last-minute rehearsal checklist	23
18. Quick glossary	24
19. Final sources to keep open during teaching	24

Quick navigation tip: use your PDF reader search for a route, file name, formula, or question number.

1. How to use this guide

The learner does not need to memorize Laravel syntax. By the end, they should be able to explain, in ordinary language:

1. What problem WISN Core addresses and what WISN means.
2. What PHP, Laravel, a database, Blade, Tailwind, Vite, Chart.js, and DomPDF each do.
3. How information travels from a form to the database and back to a page.
4. How AWT, workload standards, staff requirements, and the WISN ratio are calculated in this code.
5. What the operations module adds—and what it still does not automate.
6. What evidence supports the demo data, and which limitations must be acknowledged.

Teaching rule: after each section, ask the learner to explain it back without reading. Correct the idea first; only then show the relevant code file.

2. A two-day crash-course plan

Day 1 — Understand the project and trace the data

Time	Lesson	Teaching outcome
09:00–09:30	Project story and 60-second introduction	Learner can say what the application is for.
09:30–10:30	WISN terms and one hand calculation	Learner understands AWT, time standard, annual volume, FTE, and ratio.
10:45–11:30	Technology stack using simple analogies	Learner can name each tool and its job.
11:30–12:15	Laravel MVC and request flow	Learner can explain route → controller → service/model → view.
13:00–14:00	Database tables and relationships	Learner can describe departments and workload activities.
14:00–15:00	File-by-file guided tour	Learner can find the code that implements each feature.
15:15–16:00	Dashboard, charts, and PDF report	Learner can explain how results are displayed.

Day 2 — Explain the implementation, limits, and defend it

Time	Lesson	Teaching outcome
09:00–10:00	Walk the WISN calculator line by line	Learner can trace inputs into the returned result.
10:00–10:45	Operational staffing overlay	Learner can distinguish a shift snapshot from annual WISN.
11:00–11:45	Data provenance, privacy, and security	Learner avoids unsupported claims about real-world data or access control.
12:30–13:15	Run-through of a short live demo	Learner can demonstrate the application in a clear order.
13:15–14:15	Professor/evaluator Q&A	Learner practices short, accurate answers.
14:30–15:00	Final mock presentation	Learner answers follow-up questions without relying on the guide.

Minimum rehearsal test: the learner should be able to explain the project in 60 seconds, trace one dashboard request in 60 seconds, and do the AWT example below without assistance.

3. The project in one minute

Plain-language explanation

WISN Core is a browser-based nursing workforce-planning prototype for a hospital. A manager enters each department's available working-time components and the annual volume and time standard for workload activities. The application calculates an estimated staffing requirement and WISN ratio, then shows the results in a dashboard and downloadable PDF report. A separate operations page records manual, shift-level patient and staffing snapshots and suggests aggregate staff moves for human review. It does not connect to an EHR or automatically assign individual nurses.

Problem it addresses

Manual staffing calculations can be time-consuming, hard to repeat, and difficult for managers to interpret. The project digitizes the selected WISN calculation steps, validates common inputs, and displays a consistent department-by-department summary.

Main users

- Hospital or facility administrators reviewing overall staffing.
- Nurse managers entering department workload and staffing information.
- Academic demonstrators showing how a health-workforce method can be implemented as a web application.

What it is—and is not

It is	It is not
A Laravel web application with a server-rendered interface.	A mobile app, microservice system, or React/Vue single-page app.
A prototype that calculates staffing estimates from entered data.	A replacement for clinical judgment or workforce-policy approval.
A single-facility data model in the current repository.	A multi-hospital system with tenant isolation.
An annual/reference-period WISN workflow plus a manual operational overlay.	A live feed from patient records, staff attendance, or a roster system.
A tool that displays aggregate counts and reports.	A system that currently assigns named nurses or executes transfers.

4. WISN explained for a beginner

WISN means **Workload Indicators of Staffing Need**. It is a World Health Organization approach to estimating how many health workers are needed for a defined workload. In this project, the cadre is nursing staff and the unit of analysis is a hospital department.

The high-level question is:

Given the work a department performs, the time each task takes, and the time one nurse is available to work in a year, how many nurses are needed?

Key terms

Term	Beginner-friendly meaning
Available Working Time (AWT)	Productive hours one nurse is available to work over the chosen year, after subtracting non-working time.
Workload activity	A task included in the calculation, such as triage, medication administration, or teaching.
Time standard	The expected hours needed for one occurrence of an activity—or, for support tasks, the hours used in a shift.
Annual volume	How many times a health-service or additional activity occurs during the year.
Standard workload	The number of occurrences one nurse could perform in a year at the entered time standard.
FTE	Full-time equivalent. A value such as 1.5 means workload equivalent to one and a half full-time staff, not necessarily a specific roster.
WISN ratio	Current department staff divided by estimated required staff.
Support allowance / CAF	An adjustment for time spent on recurring support activities.
Additional activity / AAF	In this application's formula, a separate FTE contribution calculated from its annual hours.

Important time-scale distinction

The main WISN calculation uses annual workload volumes and annual available working time. It can be recalculated when a user updates those inputs, but that does not make it an intraday staffing allocator. It answers a planning question over a reference period; it does not know who is on duty right now unless operational data is separately supplied.

5. The technology stack

The stack is visible mainly in [composer.json](#), [package.json](#), [config/](#), and the [resources/](#) directory.

Tool	What it is	What it does here
PHP 8.1+	Server-side programming language.	Runs Laravel, controllers, models, and the calculation service. composer.json declares PHP ^8.1.
Laravel 10	PHP web framework.	Provides routes, middleware, validation, database access, authentication scaffolding, views, and application structure.
Composer	PHP package manager.	Installs Laravel and PHP libraries declared in composer.json / composer.lock .
MySQL (default example)	Relational database.	Stores users, departments, activities, and operational snapshots. The actual connection is set in .env ; Laravel's config also supports SQLite and other drivers.
Eloquent ORM	Laravel's object-relational mapper.	Lets code work with Department and WorkloadActivity models rather than writing routine SQL by hand.
Laravel migrations	Versioned database schema scripts.	Create and evolve tables in a repeatable way.
Blade	Laravel's server-side template engine.	Combines application data with HTML templates under resources/views/ .
Tailwind CSS 3	Utility-class CSS framework.	Styles the pages using classes such as spacing, color, layout, and responsive breakpoints.
Vite 4	Front-end development/build tool.	Loads and builds the CSS and JavaScript assets referenced by the Laravel layout.
Alpine.js 3	Small JavaScript interaction library.	Supports lightweight page interactions such as menus and toggles.
Chart.js 4.4	JavaScript charting library.	Draws dashboard charts; the dashboard loads it from a CDN.
DomPDF	PHP PDF renderer (barryvdh/laravel-dompdf).	Converts a Blade report template into a downloadable PDF.
Laravel Breeze	Authentication starter kit.	Supplies login, registration, password-reset, and profile scaffolding.
PHPUnit 10	PHP testing framework.	Runs unit and feature tests. The repository's test environment is configured to use in-memory SQLite.
Git / GitHub	Version-control tools.	Track the source and its development history.

Composer versus npm

- **Composer** handles PHP/Laravel packages.
- **npm** handles browser-side development packages such as Tailwind, Alpine.js, and Vite.
- Installing one does not install the other.

What not to overstate

[composer.json](#) also declares packages such as Sanctum and Guzzle, but the primary browser workflow is the authenticated Laravel [web.php](#) flow. Do not describe the product as API-first just because an API route file or package exists.

6. Architecture using a simple analogy

Think of a small hospital office:

Software part	Office analogy	Project examples
Route	Receptionist: decides where a request should go.	routes/web.php , routes/auth.php
Middleware	Door check: verifies the request is allowed to continue.	auth , verified route middleware
Controller	Coordinator: validates a request and asks the right part of the system to do work.	DashboardController , DepartmentController
Model / ORM	Organized record book: represents and retrieves stored data.	Department , WorkloadActivity
Service	Specialist calculator: holds business rules and calculations.	WisnCalculatorService , OperationalStaffingService
Database	Filing cabinet: persists records after a page closes.	MySQL configured by .env
View	Printed/displayed form: turns prepared data into a page.	resources/views/dashboard.blade.php
Migration	Database construction plan: creates a table consistently.	Files under database/migrations/

Request path diagram

```
Browser
-> routes/web.php
-> auth / verified middleware
-> controller
-> model and/or calculation service
-> database query or update
-> Blade view (HTML) or DomPDF (PDF)
-> response back to browser
```

This is a **Laravel monolith**: the backend, server-rendered pages, and business logic live in one application. The major user interface is not a separate frontend application.

Why put calculation logic in a service?

[WisnCalculatorService](#) keeps the WISN maths out of the page controller. The dashboard and PDF report can use the same calculation logic. It is easier to inspect and test one calculation service than to duplicate the formula in multiple views or controllers.

7. Database: what is stored and how it relates

A relational database stores related facts in tables. The key relationship is:

```
Department 1 ---- many WorkloadActivity rows
```

One department can have many workload activities; each activity belongs to one department.

Core tables

Table	What it stores	Important fields / links
users	Application login accounts.	Name, email, password hash, verification timestamp.
departments	One row per hospital department.	Name, type, current_staff , annual workdays, holidays, leave, training, and hours per day.
workload_activities	One row per task included in WISN.	department_id , name, type, time standard, annual volume. Department deletion cascades to these activities in the original schema.

Laravel also has framework tables for password-reset tokens, failed jobs, and Sanctum personal access tokens. Those are not the WISN calculation inputs.

Department fields and AWT

The department stores the components used to calculate AWT:

- [working_days_per_year](#)
- [public_holidays](#)
- [annual_leave_days](#)
- [sick_leave_days](#)
- [training_days](#)
- [working_hours_per_day](#)

AWT itself is computed by an Eloquent accessor; it is not stored as a separate current column.

An earlier migration created [available_working_time_hours](#), and a later migration removed it after the breakdown fields became the source of truth.

Activity types

Stored value	Purpose	Annual volume
health_service	Direct service activity, such as a clinical task.	Required by controller validation.
support	Recurring shift work, such as handover.	Not used by the code for the support fraction.
additional	Additional activity such as teaching.	Required by controller validation.

Operational tables added alongside WISN

Table	What it stores
operational_rounds	Timestamp, entering user, and optional round note for a manually captured facility snapshot.
department_operational_snapshots	Per-department patient count, high-acuity count, on-duty staff, locally assessed required on-duty staff, and note. One department appears at most once per round.
mobilization_recommendations	Aggregate count from a sending department to a receiving department; status, rationale, and reviewer/time fields.

The operations module uses separate [on_duty_staff](#) snapshot values. It does **not** overwrite [departments.current_staff](#), which is the WISN department headcount.

Who owns the department rows?

In this prototype, there is no [hospital_id](#) and no user-to-department ownership link. Departments are shared across accounts. This is a single-facility data model, not multi-tenant access control.

8. WISN calculation: step by step

The implementation is in [app/Services/WisnCalculatorService.php](#). Its main entry point is:

```
calculateDepartmentStaffing(Department $department): array
```

It returns an array containing the department details, AWT, health-service FTE, support multiplier, AAF FTE, total required staff, WISN ratio, status, and an activity breakdown. It does not write the calculated result back to a database table; the controller passes the returned array to a view.

Step 1 — Calculate AWT

```
AWT = (working days - public holidays - annual leave - sick leave - training) × hours/day
```

Using the sample/default values in the code:

```
(260 - 13 - 18 - 12 - 5) × 8  
= 212 × 8  
= 1,696 hours per nurse per year
```

Step 2 — Health-service standard workload and staff

For each [health_service](#) activity:

```
Standard workload = AWT / time standard  
Required FTE for activity = annual volume / standard workload
```

A larger annual volume or longer time per task increases required FTE. A higher AWT increases the amount one nurse can contribute over the year.

Step 3 — Support activities and CAF

For each [support](#) activity, the code calculates its fraction of one working day:

```
Support fraction = support hours per shift / working hours per day  
Total support fraction = sum of support fractions  
CAF = 1 / (1 - total support fraction)
```

The code uses the department's working-hours-per-day value; it does not hard-code an eight-hour shift. The multiplier is used to adjust the health-service FTE.

Step 4 — Additional activities and AAF

The current code uses this project-specific formula:

```
Annual additional hours = sum(annual volume × time standard)  
AAF FTE = annual additional hours / AWT
```

The AAF FTE is added to the health-service FTE after the CAF adjustment.

Step 5 — Total and ratio

```
Total required staff = (health-service FTE × CAF) + AAF FTE  
WISN ratio = current department staff / total required staff
```

Interpretation:

- Ratio below 1.00: current headcount is below the model's estimated requirement.
- Ratio equal to 1.00: current headcount equals the estimate.
- Ratio above 1.00: current headcount is above the estimate.

The status labels in this application are: [no_data](#) for a non-positive ratio, [critical](#) below 0.90, [borderline](#) from 0.90 up to but not including 1.00, [adequate](#) at exactly 1.00, and [surplus](#) above 1.00.

Worked example using this code's formulas

This is a teaching example, not a hospital recommendation:

1. AWT is 1,696 hours/year.
2. One health-service task takes 0.5 hours; annual volume is 3,392 tasks.
3. Standard workload is $1,696 / 0.5 = 3,392$ tasks per nurse per year.
4. Health-service requirement is $3,392 / 3,392 = 1.00$ FTE.
5. One support activity takes 0.5 hours in an 8-hour day: support fraction $0.5 / 8 = 0.0625$ (6.25%). CAF is $1 / (1 - 0.0625) = 1.0667$.
6. An additional activity occurs 1,696 times and takes 0.5 hours each: $848 \text{ hours} / 1,696 \text{ AWT} = 0.50$ AAF FTE.
7. Total required is $(1.00 \times 1.0667) + 0.50 = 1.5667$ FTE (displayed to two decimals as 1.57).
8. If current staff is 2, the ratio is approximately $2 / 1.5667 = 1.28$, so the code labels the department surplus.

Formula caveat to communicate

The guide above describes the **formula currently coded in this repository**. It should not be presented as proof that every WISN methodological assumption has been independently certified. In particular, clinical time standards, workload volumes, support allowances, additional-activity treatment, and category definitions should be checked with the relevant WISN/health-workforce experts before operational use. The current service applies CAF only when the support fraction is between 0 and 1; invalid allowance inputs should be validated and reviewed.

9. Walkthrough of the main user journeys

A. Dashboard: GET /dashboard

1. `routes/web.php` maps `/dashboard` to `DashboardController@index`.
2. The route is inside the authentication middleware group.
3. `DashboardController` queries departments and eager-loads their activities.
4. It calls `WisdCalculatorService` once for each department.
5. It adds up current and required staff to calculate a facility-level ratio: `sum(current) / sum(required)`.
This is a weighted facility ratio, not the average of department ratios.
6. It passes the result arrays to `resources/views/dashboard.blade.php`.
7. Blade renders the page, tables, cards, and Chart.js charts.

Files to show: `routes/web.php`, `app/Http/Controllers/DashboardController.php`, `app/Services/WisdCalculatorService.php`, `resources/views/dashboard.blade.php`.

B. Department management

1. The user opens a department form.
2. The browser sends a POST or PUT request.
3. `DepartmentController` validates the fields and rejects a non-positive net working-day count.
4. Eloquent writes the department row.
5. The browser is redirected to the department list.

Files to show: `app/Http/Controllers/DepartmentController.php`, `app/Models/Department.php`, `resources/views/departments/`, department migrations.

C. Workload activities

1. Activity routes are nested under a department, for example `/departments/{department}/activities`.
2. The controller validates name, type, time standard, and annual volume rules.
3. The activity is stored with a `department_id` foreign key.
4. The next dashboard/report request loads the updated activities and recalculates the WISN result; calculations are not cached in a separate results table.

Files to show: `app/Http/Controllers/WorkloadActivityController.php`, `app/Models/WorkloadActivity.php`, `resources/views/activities/`, workload-activity migration.

D. PDF report: GET `/report`

1. `ReportController` loads departments with activities.
2. It calls the same WISN service.
3. It prepares an AWT breakdown and facility totals.
4. `Pdf::loadView()` renders `resources/views/reports/wisn-summary.blade.php` through DomPDF.
5. The browser downloads the report.

The PDF view uses inline CSS because it is rendered by the server-side PDF engine rather than the browser's normal asset pipeline.

E. Operations snapshot: POST `/operations/snapshots`

1. A user enters a complete snapshot for each current department.
2. The controller validates counts and requires the high-acuity count not to exceed total census.
3. `OperationalStaffingService` writes a timestamped round and its department snapshots inside a database transaction.
4. It marks old pending proposals as superseded.
5. It finds departments above and below the manually entered required-on-duty target and creates count-level suggestions.
6. The page displays the latest snapshot and proposals.

Files to show: `OperationalSnapshotController.php`, `OperationalStaffingService.php`, operational migrations, `resources/views/operations/index.blade.php`.

10. The operational staffing overlay: current behavior

The operations feature keeps a second time horizon beside WISN:

- **Annual planning:** WISN uses annual workload and AWT to estimate a department's longer-term staffing requirement.
- **Shift snapshot:** a user manually enters current patient census, high-acuity count, on-duty staff, and the facility-approved number of nurses required now.

The application compares on-duty staff with the entered current target. It proposes aggregate movements only when one department is above its target and another is below. For a simple hypothetical example:

Unit	Patients	On duty	Required now	Balance
Medical Ward	4	6	4	+2
Surgical Ward	18	3	5	-2

The service may suggest **2 aggregate staff from Medical Ward to Surgical Ward**. A reviewer can approve or decline the count suggestion.

Safety and scope boundaries

- The `required_on_duty_staff` target is manually entered from a locally approved policy; patient count and acuity are recorded for context but are not converted into staffing by this prototype.
- The service does not choose a named nurse, check specialty competencies, check breaks or shift limits, or update a roster.
- Approval records a human decision; it does not execute a transfer or modify `departments.current_staff`.
- New snapshots supersede unreviewed recommendations. Approval is blocked when data is older than the configured freshness window (default 30 minutes), the department set changed, or the move would breach a source target or exceed a destination target.
- The app has no role/permission model beyond authenticated web access. Do not describe its approval workflow as manager-only authorization.
- Free-text notes exist. Do not enter patient names or identifiers.
- This is manual data entry, not a live EHR, patient census, or staff attendance integration.

How to describe the feature in a presentation

Say: **“We added a separate shift-level operational snapshot and aggregate recommendation workflow. It is designed not to disturb the annual WISN calculation. The current prototype still depends on a human-entered, locally approved shift target and human review; it does not yet automate individual nurse deployment.”**

Do not say: “The system is connected to live patients,” “the system knows current acuity,” or “approval automatically sends a nurse to another unit.” Those statements are not supported by the current code.

11. Where the information came from

Question	Where to look in this repository	What it tells you
What is the intended academic problem and scope?	WISN_Core_Project_Report.md , WISN_Proposal_Final (1).pdf	Background, aims, scope, and project rationale.
What is the current software architecture?	ARCHITECTURE.md	Routes, models, calculation service, views, and current operational overlay.
What does the code actually calculate?	app/Services/WisnCalculatorService.php	Exact formulas, rounding, status thresholds, and guards.
What are the table fields and relationships?	database/migrations/ , app/Models/	Database structure and Eloquent relationships.
What sample data appears in the app?	database/seeder/DatabaseSeeder.php , NepalDataSeeder.php	Demo departments, activities, and comments about data provenance.
Which PHP libraries are declared?	composer.json , composer.lock	Framework and PHP dependencies.
Which front-end tools are declared?	package.json , vite.config.js , resources/js/ , resources/css/	Vite, Tailwind, Alpine, and browser assets.
How is the browser flow routed?	routes/web.php , routes/auth.php	Page endpoints and authentication scaffold routes.
What is the original WHO reference?	9789240070066-eng.pdf and the reference list in the project report	Methodology reference material included in the repository.
What Nepal context material is included?	Nepal Health Facility Data Summary.pdf , WISN_Core_Source_Access_Sheet.pdf	Context/source documents bundled with the project.

Data provenance: an important answer

The repository contains reference PDFs and seeders whose comments describe Nepal-oriented demo assumptions and sources. The running application does not automatically query those PDFs or connect to a government database. Seeded data are sample/demo values, not proof that the application contains actual hospital records or that every entered standard is an official national standard. For an academic presentation, cite the exact source for any specific number and label simulated data clearly.

Annual turnover note

The wider project may include a separately calculated annual nurse-turnover result. In the Laravel source inspected for this guide, there is no turnover table, model, controller, or calculator service wired into the app. Present turnover as a separate result/module unless the presenter can show its actual integration point. It is a longer-term workforce-supply signal; it is not an input to the current WISN formula or the operations page.

12. Start-to-finish teaching walkthrough: exactly what to open and say

Use this as an **18-22 minute guided code tour**. It is written as presenter instructions, not just a list of files. Keep the repository open in an editor and, if the application runs on the presentation machine, keep the browser open too. On Windows/Linux use **Ctrl+P** to open a path and **Ctrl+F** to find a symbol; on Mac use **Cmd+P** and **Cmd+F**. Do not read whole files aloud: find the named class, method, route, or phrase, point to the few relevant lines, explain them, then follow the transition to the next file.

Before the walkthrough (2 minutes)

1. Open the repository root, [niranjanrimal25/wisn-core](#), in the editor.
2. Keep [ARCHITECTURE.md](#), the application dashboard (if runnable), and this guide available.
3. Use only a disposable demo database if you will show seeded data. The current main seeder deletes/replaces department and activity demo rows; do not seed a database with real operational data.
4. If the local app cannot run, do not pretend it did. You can still teach the request path using the source and the screenshots/report available to you.

Opening words: “I’ll trace one annual WISN result from the browser route, through the PHP calculation, to the page and PDF. After that I’ll show the separate shift snapshot feature, because it answers a different question.”

Part A — trace the annual WISN feature

Step 1 — Show the project map (about 1 minute)

Open: [ARCHITECTURE.md](#) at the repository root. Do not use the root [README.md](#) as the project overview; it is still the stock Laravel starter README.

Point to: the “Request flow” and “Service layer” sections. Briefly identify the route, controller, model, service, database, and Blade view. Explain that [ARCHITECTURE.md](#) is a map; the source files that follow are the evidence.

Tell them: “This is one Laravel application. A browser request comes in through a route, a controller coordinates the work, models read the stored records, a service performs the WISN maths, and a Blade view displays the result.”

Move on: “First, let’s identify the tools this application uses.”

Step 2 — Identify the tools (about 1 minute)

Open: [composer.json](#), then [package.json](#).

Find/show: In `composer.json`, point to the PHP requirement and the Laravel and `barryvdh/laravel-dompdf` dependencies. In `package.json`, point to the Vite build scripts and the Tailwind/Alpine dependencies. Chart.js is loaded from a CDN in the dashboard view, so do not say it is installed by `package.json`.

Tell them: “Composer installs the PHP/Laravel side. npm installs and builds the browser-side CSS and JavaScript. They are two separate package managers. Laravel and Blade render the page; Chart.js draws the dashboard charts; DomPDF creates the downloadable report.”

Move on: “Now let’s find the URL that starts the dashboard request.”

Step 3 — Start at the browser route (about 1 minute)

Open: `routes/web.php`.

Find/show: `Route::middleware(['auth', 'verified'])`, then the `GET /dashboard` route that points to `DashboardController::index`. Also point out the department/activity routes and `GET /report`, but stay on the dashboard route for the main trace.

Tell them: “A route connects a URL and HTTP action to PHP code. When an authenticated browser asks for `/dashboard`, Laravel sends the request to `DashboardController::index`. The routes are grouped behind authentication middleware; the email-verification caveat is explained later, so I will not claim verification is effectively enforced without checking the User model.”

Move on: “The route hands the request to the controller.”

Step 4 — Follow the dashboard controller (about 1½ minutes)

Open: `app/Http/Controllers/DashboardController.php`.

Find/show: `index()`, `Department::with('activities')->get()`, the loop that calls `calculateDepartmentStaffing($dept)`, the totals, and the final `return view('dashboard', [...])`.

Tell them: “The controller coordinates the request. It loads each department and its activities, asks the calculation service for a result, totals the facility numbers, and passes prepared data to the dashboard view. It is not where the WISN formula is written.”

Teaching check: Ask the learner, “Which line calls the maths?” They should point to `calculateDepartmentStaffing($dept)`.

Move on: “Before we inspect that maths, let’s see how the inputs get into the database.”

Step 5 — Show data entry and validation (about 1½ minutes)

Open: `app/Http/Controllers/DepartmentController.php`, then `app/Http/Controllers/WorkloadActivityController.php`.

Find/show: In `DepartmentController`, find `validationRules()`, `store()`, and the net-working-days check. In `WorkloadActivityController`, find `validationRules()` and `store()`; point out the allowed activity types and the rule that requires annual volume for `health_service` and `additional` activities.

Tell them: “These controllers accept form submissions and validate them before saving. Department inputs include the current headcount and the components used for AWT. An activity is created through its department relationship, so it is linked to the correct department. Invalid or incomplete entries should be rejected rather than silently used.”

Move on: “Those PHP model relationships describe how the saved records connect.”

Step 6 — Explain the models and database tables (about 2 minutes)

Open: `app/Models/Department.php`, then `app/Models/WorkloadActivity.php`.

Find/show: `Department::activities()` (`hasMany`), `WorkloadActivity::department()` (`belongsTo`), and `availableWorkingTimeHours()` in `Department.php`.

Tell them: “A department can have many workload activities, while each activity belongs to one department. AWT is calculated by the Department model from working days minus holidays, annual leave,

sick leave, and training, multiplied by hours per day. In the current model, AWT is a computed attribute, not a manually entered total column.”

Then open: [database/migrations/2026_05_18_044256_create_departments_table.php](#), [database/migrations/2026_05_18_044256_create_workload_activities_table.php](#), and [database/migrations/2026_05_18_add_awt_breakdown_to_departments.php](#).

Important teaching detail: the first department migration contains the earlier [available_working_time_hours](#) column. The later AWT-breakdown migration removes that column and adds the component fields. Explain that migrations run in order, so the final schema is understood by reading the later migration too—not by stopping at the first file.

Move on: “Now the service can read the department’s derived AWT and its activities.”

Step 7 — Teach the calculation service (about 3 minutes)

Open: [app/Services/WisnCalculatorService.php](#).

Find/show: `calculateDepartmentStaffing(Department $department)`. Trace the code in this order:

1. `$department->available_working_time_hours` — gets the model’s AWT value.
2. The loop over `$department->activities` — handles each activity type.
3. `health_service` — calculates standard workload, then required FTE from annual volume.
4. `support` — divides support hours by working hours per day and accumulates the support fraction.
5. `additional` — adds annual activity hours, then divides by AWT to get AAF FTE.
6. The CAF guard and formula for total required staff.
7. The WISN ratio, status method, and returned result array.

Tell them: “This service is the calculation engine. Health-service tasks, support time, and additional activities follow different code branches. The final required staff and ratio are assembled in one result array, rounded for display, and returned to the caller; this method does not save a WISN result row to the database.”

Use the worked example: return to Section 8 of this guide and calculate the labelled example on paper. Keep saying “this is the formula currently coded here,” not “this proves every methodological assumption has been clinically validated.”

Move on: “The controller passes this result array to Blade, which turns it into the screen.”

Step 8 — Show how the dashboard displays the result (about 1½ minutes)

Open: [resources/views/dashboard.blade.php](#).

Find/show: “Facility Staffing Dashboard,” the summary cards, the department table, the two `<canvas>` chart elements, and the script near the bottom that turns `@json($departmentResults)` into Chart.js datasets. Point to the report link if visible.

Tell them: “Blade is the server-side HTML template. The controller has already prepared the numbers; this view displays them as cards and a department table. The JavaScript reads the prepared results and Chart.js draws the charts. The view displays the calculation—it is not a second calculation engine.”

Move on: “The downloadable report uses the same calculation service, rather than a separate WISN formula.”

Step 9 — Follow the PDF report path (about 1 minute)

Open: [app/Http/Controllers/ReportController.php](#), then [resources/views/reports/wisn-summary.blade.php](#).

Find/show: `generate()`, the call to `calculateDepartmentStaffing()`, `Pdf::loadView('reports.wisn-summary', ...)`, and the download response. In the Blade report, point to the facility totals, department-level results, activity breakdown, and formula summary.

Tell them: “The report controller calls the same WISN service used by the dashboard, adds the AWT input breakdown, and passes the data to a Blade report template. DomPDF renders that template and Laravel returns a PDF download.”

Transition: “That finishes the annual planning path. The next files are a different feature with manually entered shift-level data.”

Part B — trace the separate operational snapshot

Step 10 — Show the operations route and screen (about 1 minute)

Open: [routes/web.php](#) and find `GET /operations` and `POST /operations/snapshots`. Then open [resources/views/operations/index.blade.php](#).

Find/show: the “Manual snapshot” label, the warning that the workflow is not a live roster, and the “Required nurses now” input. Also point to the snapshot and recommendation areas lower in the view.

Tell them: “This page is separate from annual WISN. A user manually records current census, high-acuity count, on-duty staff, and a locally approved required-on-duty target. Patient count and acuity are context here; the application does not convert them into a nurse target.”

Move on: “When the form is submitted, the snapshot controller checks it before anything is saved.”

Step 11 — Explain snapshot validation (about 1 minute)

Open: [app/Http/Controllers/OperationalSnapshotController.php](#).

Find/show: `store()`, the validation rules, the submitted-versus-current department ID check, the high-acuity count check, and the call to `recordRound()`.

Tell them: “The controller requires a complete entry for every current department and checks that high-acuity patients do not exceed the total census. It then passes validated values to the operational service. This is manual form entry, not an EHR or attendance-system import.”

Move on: “The service stores the round and compares the entered targets to suggest aggregate counts.”

Step 12 — Trace the operational service and human review (about 2 minutes)

Open: [app/Services/OperationalStaffingService.php](#).

Find/show: `recordRound()`, `DB::transaction()`, the pending-recommendation superseding update, snapshot creation, and `createRecommendations()`. Then briefly find `approve()` and the `ensureLatestRound()` / `ensureFresh()` safeguards.

Tell them: “This service records one timestamped facility round and all department snapshots in a transaction. It compares only the manually entered on-duty count and required-now target, then creates aggregate count suggestions. Approval records a reviewer decision and checks that the snapshot is current and the move stays within the recorded targets. It does not select a named nurse, change `departments.current_staff`, or update a roster.”

Move on: “The database has separate operational tables to preserve that separate time horizon.”

Step 13 — Show operational storage, display, and tests (about 2 minutes)

Open: these migrations in order:

- [database/migrations/2026_10_05_000001_create_operational_rounds_table.php](#)
- [database/migrations/2026_10_05_000002_create_department_operational_snapshots_table.php](#)
- [database/migrations/2026_10_05_000003_create_mobilization_recommendations_table.php](#)

Tell them: “A round records when and who entered the snapshot. Per-department snapshot rows keep counts for that round. Recommendation rows keep the sending/receiving department, aggregate staff count, status, rationale, and review information. These are not patient-level records and they do not overwrite the annual WISN headcount.”

Then open: [app/Http/Controllers/OperationalDashboardController.php](#) and return to [resources/views/operations/index.blade.php](#). Point out that the dashboard controller loads the latest round and that Blade displays the snapshot and review status.

Finally open: `tests/Unit/WisnCalculatorServiceTest.php` and `tests/Feature/OperationalStaffingTest.php`. Show one assertion that checks a known WISN result and one that verifies snapshot recommendations do not change WISN/headcount, plus the stale-snapshot test if time allows.

Tell them: “Tests show what behavior the developers intended to protect. Their presence does not prove the test suite passed on this machine; we should run `php artisan test` in a configured Laravel environment before claiming a passing run.”

Finish the walk-through (about 1 minute)

Close the editor and return to the dashboard or your presentation slide. Say:

“The annual WISN path is entered department/activity data → controller → calculation service → dashboard or PDF. The operational path is a separate manual shift snapshot → aggregate suggestion → human review. The next engineering steps are validating local standards, improving access control, and connecting trustworthy data sources safely; the current prototype does not make clinical staffing decisions automatically.”

Ask the learner to trace both paths back without reading. If they can say what each file does and what the system does **not** do, they are ready to present.

README warning: the root `README.md` is still the stock Laravel starter README. Use `ARCHITECTURE.md`, the project report, and the code—not the starter README—as the project-specific sources.

13. How to run a local demonstration

The person presenting should rehearse in the same environment they will use on presentation day. A typical local setup needs PHP 8.1+, Composer, Node/npm, and a configured database.

1. Clone the repository.
2. Copy `.env.example` to `.env` and set the database connection.
3. Run `composer install`.
4. Run `php artisan key:generate`.
5. Run `npm install`, then `npm run build` (or `npm run dev` during development).
6. Run `php artisan migrate`.
7. Seed only a disposable demo database if sample data are needed.
8. Run `php artisan serve` and open the local URL in a browser.
9. Log in, then demonstrate Dashboard, Departments, Activities, Operations, and Report.

Typical commands:

```
composer install
cp .env.example .env
php artisan key:generate
npm install
npm run build
php artisan migrate
php artisan db:seed
php artisan serve
php artisan test
```

Seed warning: the current `DatabaseSeeder` deletes existing workload activities and departments before inserting its demo set. Run it only against a disposable local/demo database. Do not run it against real operational data. Departments with operational history are protected from hard deletion in the application.

Demo order (about 4-6 minutes)

1. **Start with the dashboard.** Explain current staff, required staff, ratio, status, and chart.
2. **Open Departments.** Show one department's AWT components; explain AWT is derived, not manually stored as one number.
3. **Open its Activities page.** Show the three categories and explain annual volume/time standard.
4. **Return to Dashboard.** Point out that changing saved inputs causes the calculation to be recomputed on the next request.
5. **Open Report.** Download the PDF and show that it summarizes the same calculation service.
6. **Open Operations.** Use clearly labelled hypothetical counts. Save a full department snapshot, show an aggregate suggestion, and explain that it does not assign an individual nurse.
7. **Close with limitations and next steps.** This demonstrates technical confidence rather than hiding prototype boundaries.

Avoid entering patient names or identifiers. If a department is not in the latest complete snapshot, or the round is stale, refresh the snapshot before attempting an approval.

14. Security, testing, and quality questions

What the code does

- Routes are grouped under Laravel's `auth` and `verified` middleware names.
- Controllers validate input before saving it.
- Laravel web forms use CSRF protection through the web middleware stack.
- Eloquent parameterizes database operations; Blade's `{{ }}` output is escaped by default.
- WISN logic is separated into a service and now has a regression test.
- The operational approval is transaction-based and checks that a count move does not cross recorded targets.
- Operational text inputs warn users not to store patient identifiers.

Important access-control caveat

The current `User` model has its `MustVerifyEmail` implementation commented out. Laravel's `verified` middleware only enforces verification for models that implement the verification contract. Therefore, do not claim email verification is effectively enforced without checking/fixing that model behavior. Also, the app has no manager/admin role authorization: authenticated users can reach the operational review routes in this prototype.

Tests

The repository includes standard Breeze authentication/profile tests, a WISN regression test, and operational feature tests for snapshot validation, recommendations, approval behavior, stale rounds, superseding, and department-history protection. Run:

```
php artisan test
```

A test file existing is not the same as a passing test run. Before the presentation, run the suite on a machine with dependencies and a working database driver; report honestly whether it passed.

15. Evaluator questions and ready-to-say answers

Use the answer in quotes as the first response. Add detail only if the evaluator asks for it.

Project and method

Q1. What is WISN Core?

"It is a Laravel web prototype that calculates and displays nursing staffing estimates by hospital department using entered workload volumes, time standards, and available working time."

Q2. What problem does it solve?

"It makes selected WISN calculations repeatable and easier to interpret than doing each calculation manually in a spreadsheet. It gives managers a dashboard and report for reviewing staffing estimates."

Q3. What does WISN stand for?

"Workload Indicators of Staffing Need, a WHO methodology for estimating health-worker requirements from workload and time standards."

Q4. Is the application itself the WHO methodology?

"No. WISN is the methodology; this application is a software implementation of selected calculations and workflows. The underlying standards and assumptions still need appropriate expert validation."

Q5. Why is annual data used?

"The core WISN calculation compares annual workload with annual available working time. It is useful for workforce planning over a reference period, not for choosing a nurse to move this minute."

Q6. What does a WISN ratio of 0.8 mean?

"It means current staff is about 80 percent of the calculated requirement under the entered inputs. The app labels values below 0.90 as critical, but the ratio is an estimate—not an automatic clinical order."

Q7. What does a ratio above one mean?

"The department's entered current headcount is above its estimated workload-based requirement. It does not automatically mean that a particular nurse is safe or available to redeploy."

Q8. Is the data real?

"The repository contains demo/seed data and reference material. The project scope describes simulated data. I would label demo values as simulated and cite a source before presenting any individual figure as official."

Technology and architecture

Q9. What is Laravel?

"Laravel is a PHP web framework. It gives the project a structured way to define routes, controllers, database models, validation, authentication, and page templates."

Q10. Why Laravel instead of plain PHP?

"Laravel provides standard tools for routing, database migrations, ORM access, validation, authentication, and templating, which makes a small application easier to organize and maintain."

Q11. What is MVC?

"MVC separates concerns: models represent stored data, views render the interface, and controllers coordinate requests. The WISN math is additionally kept in a service class."

Q12. What happens when someone opens the dashboard?

"The web route sends the request to `DashboardController`; it loads departments and activities, calls `WISNCalculatorService`, then passes the results to a Blade view that renders the cards, table, and charts."

Q13. What is Eloquent?

"Eloquent is Laravel's ORM. It maps database rows to PHP model objects and defines relationships such as one department having many workload activities."

Q14. What are migrations?

"Migrations are version-controlled database schema changes. They let a developer create or update tables in a repeatable way with `php artisan migrate`."

Q15. Why use Blade?

"Blade renders HTML on the server and can display the data prepared by Laravel. That fits this CRUD-and-dashboard prototype without needing a separate JavaScript frontend application."

Q16. What does Vite do?

"Vite serves front-end assets during development and builds CSS/JavaScript for deployment. The Laravel layout loads the app CSS and JavaScript through the Vite integration."

Q17. Which tool draws the charts?

"Chart.js draws the dashboard charts. In this project it is loaded from a CDN on the dashboard page."

Q18. How is the PDF report made?

"`ReportController` calls the same WISN calculation service, passes the results to a Blade report template, and DomPDF renders that template as a downloadable PDF."

Q19. Is it a single-page app or API-first system?

"No. The main interface is a server-rendered Laravel web application using Blade. The core user flow is in `routes/web.php`."

Data and calculation

Q20. Where is AWT stored?

"Its components are stored on the department record. The total is calculated by an Eloquent accessor, so it is not a separately entered AWT column in the current schema."

Q21. What is a time standard?

"It is the entered time associated with an activity. For health-service and additional activities it is time per occurrence; for support activities the code treats it as hours per shift."

Q22. How does the activity table relate to departments?

"A workload activity stores a department foreign key. One department can have multiple activities, and each activity belongs to one department."

Q23. Does the database store every calculated ratio?

"The dashboard calculation is generated when the request is processed and passed to the view. The calculated result is not stored as a separate WISN-results table in this implementation."

Q24. How is the facility ratio calculated?

"The dashboard divides the sum of current staff by the sum of required staff across departments. It is not a simple average of the department ratios."

Q25. Where does the demo data come from?

"The seeders contain the demo rows and comments about their assumptions and cited context. The bundled proposal, report, WHO material, and Nepal source documents provide references. I would verify a specific value against its cited source before calling it official."

Operations, safety, and limitations

Q26. Does this application now work in real time?

"Not from a live data feed. The operations page records a manually entered, timestamped snapshot. It is an initial operational decision-support layer, not an EHR or roster integration."

Q27. Does patient count automatically determine required nurses?

"No. Patient and high-acuity counts are stored for context, but the current prototype asks the user to enter the locally approved required-on-duty target. We need validated facility rules before automating that calculation."

Q28. Does approving a suggestion move a nurse?

"No. It records approval of an aggregate count suggestion. The application does not select an individual, verify competencies, or update a roster."

Q29. What prevents a suggested move from making the source unit fall below its target?

"The service calculates surplus only above the entered source target and checks cumulative approved counts again during approval. It also checks the receiving target and rejects stale snapshots."

Q30. Does it know whether a medical nurse is qualified for surgery or ICU?

"No. There is no individual nurse competency or roster model in this version. A human must verify suitability; adding a qualified roster and clinical rules is future work."

Q31. Why not use WISN alone to redeploy staff during a shift?

"WISN uses a longer planning period and annual workload inputs. Shift redeployment needs current census, acuity, actual on-duty staff, skills, and safety rules. The project keeps those horizons separate."

Q32. Is this multi-hospital?

"No. It is a single-facility prototype. The current user accounts share the same department data; there is no Hospital model or tenant boundary."

Q33. Are patient identities stored?

"The operational schema has aggregate counts, not patient identifier fields. It has free-text notes, so the UI warns users not to enter names or identifiers."

Q34. Are only managers allowed to approve?

"Not yet. The routes require authenticated web access, but this code has no manager-role authorization. We would add roles/permissions before real operational deployment."

Q35. Is email verification enforced?

"The routes use Laravel's `verified` middleware name, but the current User model has the `MustVerifyEmail` implementation commented out. I would not claim verification is enforced until that is checked or corrected."

Q36. How is annual turnover used?

"Turnover is a separate longer-term workforce-supply measure. I do not see a turnover table or service in this checked-in Laravel source, so it is not currently feeding the WISN calculation or operations recommendations."

Q37. What are the main limitations?

"The data are manually entered/demo-oriented, the model covers one facility, WISN inputs need local validation, the operational target is manual, there is no roster/skills integration, and role-based authorization is not implemented."

Q38. What would you build next?

"First validate staffing rules with clinical leadership. Then add role-based manager access, a nurse roster and competency model, import reliable patient-flow and attendance data, show data freshness, and run the recommendation feature in shadow mode before operational use."

Q39. How do you know the calculations are correct?

"The formula is isolated in a service and a regression test checks a known example. Before claiming clinical validation, I would run the test suite and compare representative outputs with independently checked manual calculations and expert-approved inputs."

Q40. Can this make staffing decisions automatically?

"No. It is a decision-support prototype. A manager reviews the output, and no recommendation should override patient safety policy or professional judgment."

16. Short presentation script

Opening (about 45-60 seconds)

"Our project is WISN Core, a Laravel-based web prototype for nursing workforce planning. It implements selected WHO WISN calculations using department working-time components, annual workload volumes, and activity time standards. The system calculates staff requirements and a WISN ratio, then presents results in a dashboard and a PDF report. We also built a separate manual shift-snapshot feature that can suggest aggregate moves for human review. It does not connect to a live hospital system or assign individual nurses, so our focus is an academic prototype and a foundation for future validation and integration."

Formula explanation (about 30 seconds)

“First, we calculate annual available working time for one nurse. For each health-service activity, we divide AWT by the time standard to get the standard workload, then divide annual volume by that standard workload to get required FTE. We adjust for support time with CAF, add the code's additional-activity FTE, and compare the total with current staff using the WISN ratio.”

Closing (about 20 seconds)

“The key contribution is making these inputs and calculations visible and repeatable through a browser-based tool. The next step is validating local standards and connecting trustworthy roster and patient-flow data while keeping clinical approval and safety rules in control.”

17. Last-minute rehearsal checklist

Explain

- [] I can say what WISN Core does in one minute.
- [] I know the difference between WISN and the manual operations overlay.
- [] I can define AWT, time standard, annual volume, FTE, CAF, AAF, and WISN ratio.
- [] I can calculate the 1,696-hour AWT example.
- [] I can explain why the ratio below 1 means the entered current headcount is below the estimated requirement.

Show

- [] I know how to log in and open the correct screens.
- [] The demo database is prepared and contains clearly labelled sample data.
- [] I can show a department, its activities, dashboard result, and PDF report.
- [] I can demonstrate Operations with hypothetical data and state that it is not a live feed.
- [] I have a backup screenshot/PDF if the network or local database fails.

Defend honestly

- [] I will call seed values simulated/demo values unless I can verify their source.
- [] I will not claim an EHR integration, automatic deployment, or clinical validation that is not in the code.
- [] I know all users currently share department data and there is no manager-role authorization.
- [] I will not enter or show patient names/identifiers.
- [] I ran `php artisan test` on a working environment—or will say clearly if I could not run it.
- [] I have checked the email-verification caveat before claiming verified accounts.

18. Quick glossary

Word	Meaning in this project
AWT	Available Working Time; annual hours available per nurse after deducted leave/non-working time.
Blade	Laravel's server-side HTML template language.
CAF	Category Allowance Factor; in this code, multiplier for the sum of support-time fractions.
Controller	Class that receives a web request, validates/orchestrates it, and returns a response.
Eloquent	Laravel's ORM for reading/writing related database records.
FTE	Full-time equivalent, a workload-sized staffing unit.
Migration	Versioned script that creates or changes database tables.
Middleware	Request filter such as authentication.
Model	PHP class representing a database entity/table.
ORM	Object-relational mapper: links code objects to relational database rows.
Route	URL-to-controller mapping.
Seeder	Script that inserts example or initial database rows.
Service	Class that holds application/business logic, such as the WISN calculation.
Time standard	Entered time used for an activity in the staffing calculation.
WISN ratio	Current staff divided by the project's calculated requirement.

19. Final sources to keep open during teaching

- [ARCHITECTURE.md](#) — implementation architecture and the new operational overlay.
- [WISN_Core_Project_Report.md](#) — project rationale, objectives, methodology context, and design discussion.
- [WISN_Proposal_Final \(1\).pdf](#) — proposal and academic framing.
- [9789240070066-eng.pdf](#) — WHO WISN reference included in the repository.
- [Nepal Health Facility Data Summary.pdf](#) and [WISN_Core_Source_Access_Sheet.pdf](#) — Nepal context/source material bundled with the project.
- [composer.json](#) and [package.json](#) — authoritative dependency manifests.
- [routes/web.php](#), [app/Services/WisnCalculatorService.php](#), models, migrations, controllers, and views — authoritative current implementation.

Final coaching reminder: confident presenters do not claim that a prototype does more than it does. Explain the method, show the code path, label the sample data, and describe the limitations as planned engineering work.